

Notas de Aula – Guilherme Sipahi – Arquitetura de Computadores

Ciclo de instruções / *Pipelining*

Ciclo de instruções

Subciclos de um ciclo de instruções:

- Busca: traz a próxima instrução até a CPU
- Execução: interpreta o OPCODE e executa a instrução
- Interrupção: se estão habilitadas as interrupções e uma interrupção aconteceu, salva o estado do processo corrente e processa a interrupção

A execução de uma instrução pode envolver um ou mais operandos da memória, cada um necessitando um acesso. Caso haja interrupção, um novo subciclo é incorporado ao ciclo: a busca do operando, após a busca do endereço.

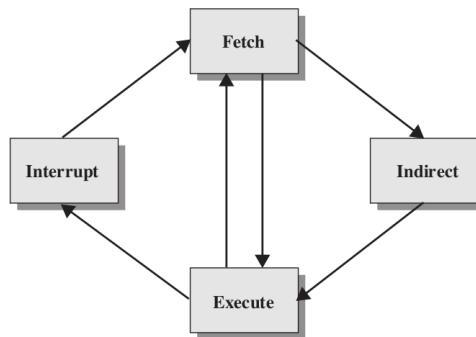


Fig 12.4 (Stallings): Ciclo de instruções

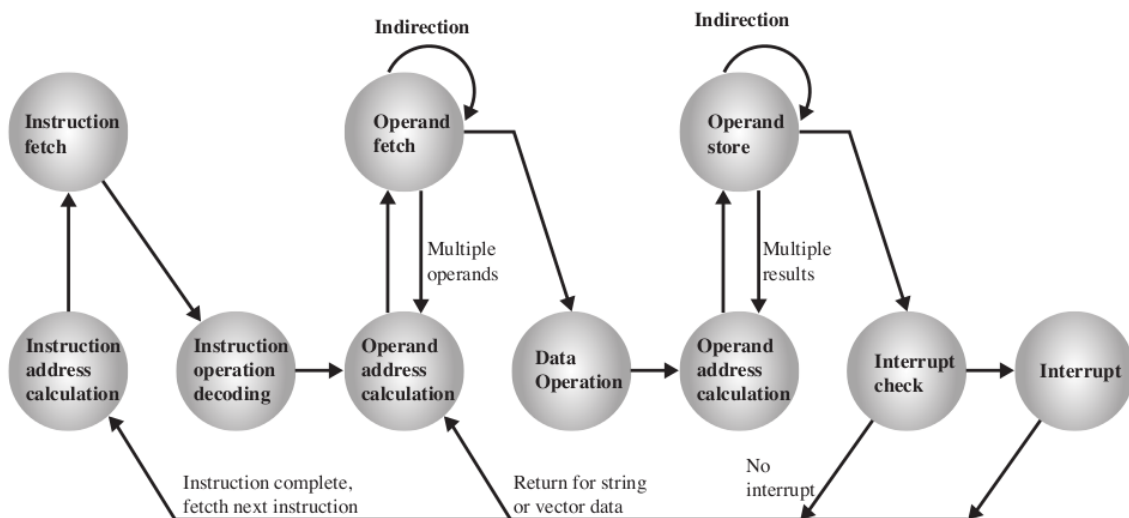


Fig 12.5 (Stallings): Ciclo de instruções detalhado

Subciclo de busca

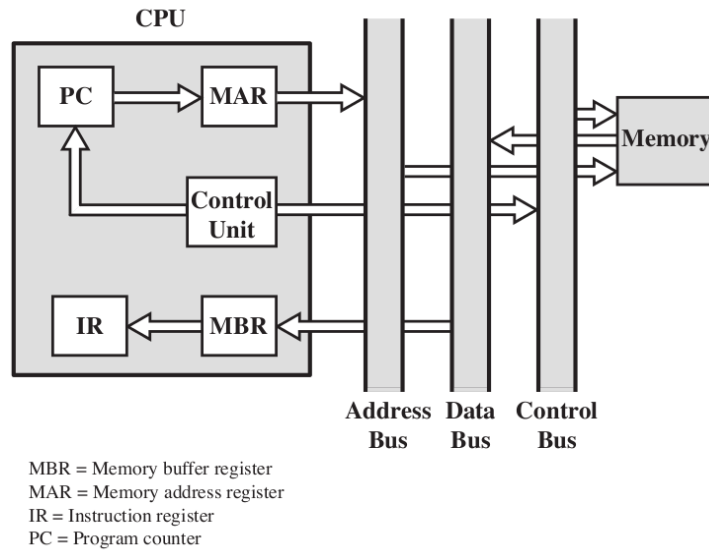


Fig 12.6 (Stallings): Fluxo de dados: Busca

- 0: PC → instrução a ser buscada
- 1: CU → PC → (MAR) → Address Bus → Memória
- 1: CU → Control Bus → Memória → Data BUS → MBR → IR
- 2: Incrementa PC

Acabado o ciclo, CU examina o conteúdo de IR para determinar se contém um operando referenciado indiretamente. Caso haja, o subciclo de indireção é executado.

Subciclo de indireção

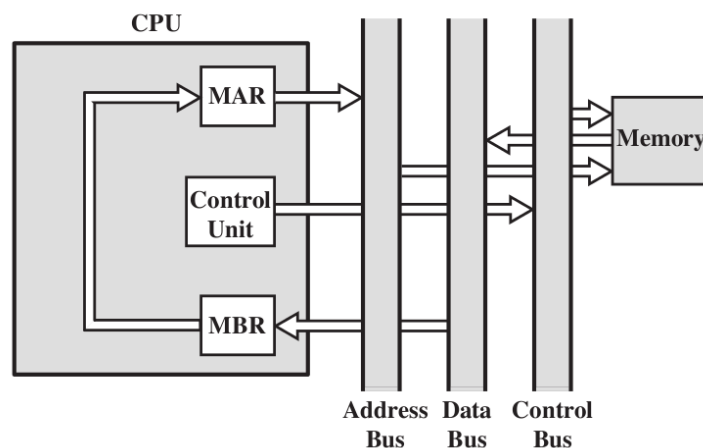


Fig 12.7 (Stallings): Fluxo de dados: Indireção

- 1: MBR(endereço) → MAR → Address Bus → Memória
- 1: CU → Control Bus → Memória
- 2: Memória → Data Bus → MBR

Subciclo de execução

O ciclo de execução não é tão simples de ser analisado pois a diversidade de instruções e de número de operandos é muito grande

Subciclo de interrupção

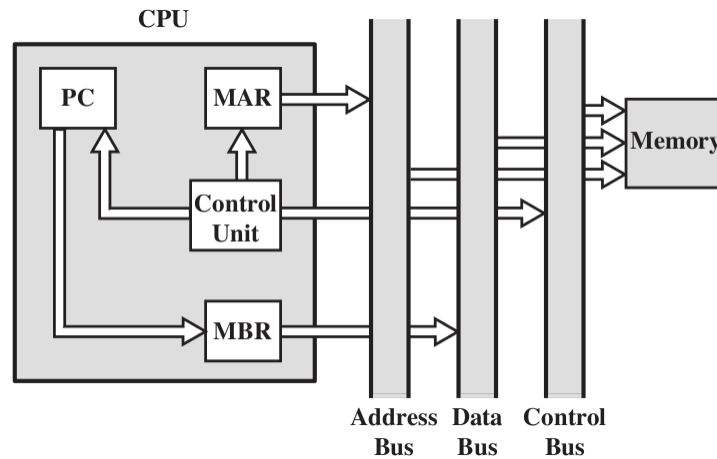


Fig 12.8 (Stallings): Fluxo de dados: Interrupção

- 1: CU → MAR → Address Bus → Memória
- 1: CU → Control Bus → Memória
- 2: CU → MBR → Data Bus → Memória

Pipeline

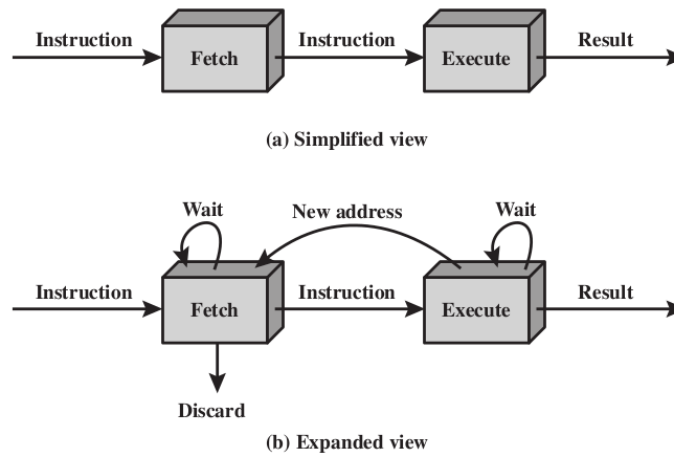


Fig 12.9 (Stallings): Instruction Prefetch

Pipeline de 2 passos (Instruction Prefetch ou Fetch Overlap):

- 1) Busca
- 2) Execução

Se os dois estágios demorassem o mesmo: *Speed Up* de 2. Mas por que não é 2?

- execução é mais longa que a busca da instrução. A primeira envolve diversas buscas.
- Branching torna o endereço da próxima instrução imprevisível devendo esperar pelo fim da execução para buscar a nova instrução.

Para aumentar o *speed up* são necessários mais estágios. Decompondo o processamento podemos ter:

- Fetch instruction (**FI**)
- Decodificação (**DI**)
- Cálculo de operandos (**CO**)
- Busca dos operandos (**FO**)
- Execução da instrução (**EI**)
- Escrita do operando (**WO**)

Time →

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instruction 1	FI	DI	CO	FO	EI	WO								
Instruction 2		FI	DI	CO	FO	EI	WO							
Instruction 3			FI	DI	CO	FO	EI	WO						
Instruction 4				FI	DI	CO	FO	EI	WO					
Instruction 5					FI	DI	CO	FO	EI	WO				
Instruction 6						FI	DI	CO	FO	EI	WO			
Instruction 7							FI	DI	CO	FO	EI	WO		
Instruction 8								FI	DI	CO	FO	EI	WO	
Instruction 9									FI	DI	CO	FO	EI	WO

Time → ← Branch Penalty

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instruction 1	FI	DI	CO	FO	EI	WO								
Instruction 2		FI	DI	CO	FO	EI	WO							
Instruction 3			FI	DI	CO	FO	EI	WO						
Instruction 4				FI	DI	CO	FO							
Instruction 5					FI	DI	CO							
Instruction 6						FI	DI							
Instruction 7							FI							
Instruction 15								FI	DI	CO	FO	EI	WO	
Instruction 16									FI	DI	CO	FO	EI	WO

Figs 12.10 e 12.11 (Stallings): Pipeline

9 instruções sem pipeline → 54 ciclos

9 instruções com pipeline → 14 ciclos (Fig 12.10)

Lidando com desvios

Em pipelines, o fluxo de instruções **deve** ser mantido constante. Desvios atrapalham (ver figuras acima). Então existem diversas maneiras de se lidar com o problema, como por exemplo:

- Múltiplos streams
- Prefetch do alvo do desvio
- Loop buffer
- Predição de desvio
- Retardamento de desvio

Múltiplos streams (Força bruta)

- Replica as porções iniciais do pipeline e busca as duas instruções.

Problemas

- com múltiplos pipelines existem atrasos no acesso aos registros e à memória
- novos desvios necessitam novas linhas

Prefetch do alvo do desvio

- Busca o alvo do desvio além da instrução subsequente. O alvo é salvo até o fim da execução do desvio.

Loop buffer

- Pequena memória de altíssima velocidade mantida pelo estágio de busca do pipeline
- Contém as n instruções mais recentemente buscadas em sequência
- Se o desvio for tomado, o hardware checa se o alvo está dentro do buffer. Caso esteja, esta instrução será buscada do buffer.

Benefícios:

- com o uso de prefetching, o buffer conterá algumas instruções sequencialmente à frente da instrução corrente. Desta forma, instruções em sequência serão buscadas sem acesso à memória.
- Se um desvio tiver como alvo uma instrução poucos endereços à frente, ela estará no buffer. Útil para sequências **IF-THEN** e **IF-THEN-ELSE**.
- Esta estratégia é particularmente útil para se lidar com loops. Se o buffer é grande o suficiente para conter todas as instruções de um loop, então estas instruções devem ser buscadas uma única vez. Nas interações subsequentes, todas as instruções estão no buffer.
- O loop buffer é muito semelhante à memória cache, diferindo no fato de guardar as instruções sequencialmente e ser muito menor.

Retardo do desvio

- É possível melhorar a performance do pipeline rearranjando automaticamente as instruções em um programa, de maneira que as instruções de desvio ocorram depois de avaliada a comparação do desvio.

Predição de desvio

Várias técnicas podem ser usadas para se prever se um desvio será tomado. Entre as mais comuns pode-se citar:

- Desvio sempre tomado
- Desvio nunca tomado
- Predição pelo OPCODE
- Chave de desvio
- Tabela de histórico de desvio

Os três primeiros *approaches* são estáticos, não dependem do histórico da execução até a instrução do desvio condicional. Os dois últimos são dinâmicos: dependem do histórico de execução:

- Estudos que analisaram o comportamento de programas mostraram que uma escolha é feita com mais frequência que a outra (em particular que tomar o desvio era a escolha mais provável). Desta maneira, se o custo de buscar qualquer dos caminhos for o mesmo, então fazer a busca do alvo do desvio sempre deve ter melhor performance.
- No entanto, em máquinas com paginação, fazer a busca do desvio tem maior possibilidade de causar um page fault que buscar a próxima instrução sequencialmente.
- A escolha de tomar ou não um desvio pode aumentar a taxa de acerto para até 75%.

Chave de desvio

- Toma a última decisão. Se o desvio foi tomado, toma-o. Caso contrário, não o toma.

Tabela de histórico de desvio

- Pequena memória cache que guarda:
 - o endereço do desvio
 - bits que descrevem o estado do desvio da instrução
 - informações do alvo do desvio. Em alguns casos o endereço da instrução e em outros a própria instrução

Pipeline do 486

Cinco estágios:

- Busca – **F**
 - opera independentemente dos outros estágios para manter os buffers de prefetch cheios.
- Decodificação (estágio 1) - **D1**
 - atua sobre os OPCODES e o modo de endereçamento. Até 3 bytes são passados do prefetch buffer para **D1**
- Decodificação (estágio 2) - **D2**
 - captura o resto da instrução (deslocamento e dados imediatos). OPCODES são transformados em sinais de controle. Controla também o cálculo de modos de endereçamento mais complexos.
- Execução - **EX**
 - operações da ALU, acesso ao cache e atualização de registro.
- Escrita – **WO**

Througput do pipeline:

		Clock cycle								
		1	2	3	4	5	6	7	8	9
Instrucion	I1	FI	DI	FO	EI	WO				
	I2		FI	DI	FO	EI	WO			
	I3			FI	DI	FO	EI	WO		
	I4				FI	DI	FO	EI	WO	

(a) Five-stage pipeline, ideal case

		Clock cycle								
		1	2	3	4	5	6	7	8	9
Instrucion	I1	FI	DI	FO	EI	WO				
	I2		FI	DI	FO	EI	WO			
	I3			Idle	FI	DI	FO	EI	WO	
	I4					FI	DI	FO	EI	WO

(b) I1 source operand in memory

		Clock cycle									
		1	2	3	4	5	6	7	8	9	10
ADD EAX, EBX		FI	DI	FO	EI	WO					
SUB ECX, EAX			FI	DI	Idle		FO	EI	WO		
I3				FI			DI	FO	EI	WO	
I4							FI	DI	FO	EI	WO

c) Data Hazard

Figs 12.16 e 12.17 do Stallings